

ProofText 语法

ProofText 是 Proof-Script 的 `#text` 命令内部使用的轻量级标记语言，用于编写随 Lean 模块一同导出的语义化正文、块级结构和行内格式。

ProofText 与 Typst 有些相似，但它是一门独立且刻意保持精简的语言。例如，`*文本*` 表示粗体，标题以 `=` 开头，无序列表项以 `+` 开头。

本文档描述当前 ProofText 解析器实际实现的语法。

1. 使用 `#text`

该命令只接受一个 Lean 字符串字面量：

```
#text "一个段落。"
```

多行内容通常最适合使用原始字符串：

```
#text r#"
= 引言 <introduction>
```

```
这是一段包含 *重要内容* 和行内公式  $x + y$  的文字。
"#
```

Lean 会先解码字符串，再由 ProofText 解析其中的内容。因此可能存在两层转义：

1. Lean 字符串字面量转义。
2. ProofText 行内转义。

在普通 Lean 字符串中，要传给 ProofText 的反斜杠本身也必须转义。因此，以下两个命令会产生相同的 ProofText 内容：

```
#text "\\*字面星号"
#text r"\*字面星号"
```

对于多行文字、LaTeX 以及包含大量 ProofText 转义符的内容，建议使用 Lean 原始字符串。如果内容本身包含当前原始字符串的结束序列，可以增加两侧 `#` 的数量。

每条成功的 `#text` 命令都会向当前页面添加一个文本组件。组件按源码顺序保存，且必须出现在 `#paper_export` 之前。

2. 完整示例

```
#text r#"
= ProofText 概览 <overview>
```

这一段包含 `*粗体*`、`_强调_`、`*_粗体强调_*`、`~~删除线~~`、`__下划线__`、`==高亮==`、`x^2^`、`a~n~`、``行内代码``、`$x + y$` 和[一个链接](https://example.com)。
另请参阅 @<overview>、@thm:main-result、@fig:diagram 和 @table:values。

```
+ 第一项
+ 第二项
```

```
1. 第一步
2. 第二步
```

术语 :: 术语的定义。

```
> 一段带有 _行内格式_ 的引用文字。
```

```
$$
\int_0^1 x^2\,dx = \frac{1}{3}
$$
```

```
```lean
theorem example : True := by
 trivial
```
"#
```

3. 源码归一化与块结构

ProofText 会先归一化缩进，使多行字符串可以与周围的 Lean 代码对齐：

- 如果首行为空，则移除一个空白首行；如果末行为空，则移除一个空白末行。
- 计算所有非空行的最少前导 ASCII 空格数，并从每一行移除相同数量的字符。
- 制表符不计入公共缩进。
- 空行用于分隔块，不会产生输出节点。

- 空字符串或只包含空白的字符串是合法的，其结果不包含任何块。

例如，归一化后，以下标题和段落会从第零列开始解析：

```
namespace Example
```

```
    #text r#"
```

```
    = 标题
```

```
    一个段落。
```

```
    "#
```

```
end Example
```

移除公共缩进后，块标记对列位置敏感。如果标记前仍有空格或制表符，该行通常会被解析为普通段落文本。

导出的页面 JSON 会保留解码后的原始字符串，而构建语义块树时使用的是归一化后的内容。

4. 块级语法

4.1 段落

连续的普通文本行构成一个段落：

```
这是同一个段落，
```

```
延续到了下一行源码。
```

段落内部的源码换行会作为换行字符保留，不会被转换为空格。当前解析器也不会生成单独的软换行或硬换行节点。

空行或另一个可识别块的开始会结束当前段落。行内语法会在完整段落上进行解析。

4.2 标题

标题由一至六个 `=` 字符和一个字面空格开头：

= 一级标题
== 二级标题
=== 三级标题
==== 四级标题
===== 五级标题
===== 六级标题

标题文字支持所有行内语法。标题正文首尾的 ASCII 空白会被移除。

可以在末尾添加可选的引用标签：

== 主要结果 <main-result>

标签必须以字母开头，之后只能包含字母、数字、连字符或下划线：

[A-Za-z][A-Za-z0-9_-]*

合法标签包括 `overview`、`chapter-2`、`result_1` 和 `A`。标签不能包含空格或点号，也不能以数字或标点开头。

命令 `elaboration` 时会注册标题标签。在同一页面或模块中重复使用标题标签会报错。标题引用使用 `@<label>`，详见第 5.7 节。

如果处于块起始位置的行以 `=` 开头，却包含超过六个 `=`、缺少必需空格或以错误标签结尾，则会报错。

4.3 无序列表

连续的 `-` 开头行构成一个无序列表：

- 第一项
- 包含 **粗体** 的第二项

每个列表项只能占一行，并作为带有行内格式的单个段落解析。目前不支持多行列表项、续行或嵌套列表。

* 不会创建无序列表。

4.4 有序列表

普通有序列表项以 `+` 开头：

- + 第一项
- + 第二项

`n.` 是强制序号语法，会把该项的 `ordinal` 设为 `n`。其后的 `+` 项由渲染器从该序号之后继续编号：

- + 第一项
- 4. 强制为第四项
- + 自动成为第五项

有序列表 JSON 的每一项都包含 `ordinal : null | Nat`；`+` 对应 `null`，`n.` 对应显式数字。与无序列表相同，每一项都是单独一行并进行行内解析，不支持多行或嵌套列表项。

4.5 定义列表

包含 `::` 的行会创建一个术语与定义的配对：

幺半群 `::` 带有结合运算和单位元的类型。

确切的分隔符是两侧各有一个字面空格的 `::` 。术语和定义会分别作为行内内容解析。如果一行含有多个分隔符，第一个用于分隔术语，其余分隔符保留在定义中。

目前每一行都会创建一个只含一项的定义列表块；连续的定义行不会合并为同一个语义定义列表。

4.6 块引用

连续的 `>` 开头行构成一个引用块：

- > 这是一段引用文字。
- > 它延续到了下一行源码。

从每一行移除一个 `>` 前缀后，引用正文会作为完整 ProofText 递归解析。因此，其中可以包含段落、标题、列表、代码块、展示公式和嵌套引用。例如：

```
> == 引用中的标题
>
> + 引用中的列表项
> + 另一项
```

属于外层引用的每个物理行都必须带有 `>` 前缀。没有标记的空行会结束引用。只包含 `>` 的行不是引用标记，因为其后必须有一个空格。

4.7 展示公式

展示公式块由内容恰好为 `$$` 的行包围：

```
$$
\sum_{i=1}^n i = \frac{n(n+1)}{2}
$$
```

开始行和结束行都必须只包含 `$$`，不能有前导或尾随空白。正文会原样保留，不会处理行内语法或 `ProofText` 转义。之后第一个内容为 `$$` 的行会结束该块。缺少结束行会报错。

行内数学公式应使用 `$...$`。

4.8 围栏代码块

代码块以三个反引号开头，并在之后第一个以三个反引号开头的行结束：

```
```lean
theorem example : True := by
 trivial
```
```

开始反引号后的文本会被去除首尾空白，并保存为可选的语言名称。不带语言的围栏会创建没有语言标记的代码块：

```
```
原样文本
```
```

正文会原样保留，不作为行内 `ProofText` 解析。结束围栏采用前缀匹配：只要一行以三个反引号开头，即使后面还有字符，也会结束代码块。目前不支持嵌套围栏；缺少结束围栏会报错。

5. 行内语法

行内结构从左向右识别，不要求单词边界或周围空白，因此 x^2 等结构可以直接出现在普通文本中。

5.1 文本格式

| 语法 | 含义 | 示例 |
|--------------------|------|---------------------------|
| *文本* | 粗体 | 这是 *重要内容* 。 |
| <u>_文本_</u> | 强调 | 这是 <u>_强调内容_</u> 。 |
| *_文本_* | 粗体强调 | 这是 *_非常重要的内容_* 。 |
| ~~文本~~ | 删除线 | 这是 ~~过时内容~~ 。 |
| <u>__文本__</u> | 下划线 | 这是 <u>__带下划线的内容__</u> 。 |
| ==文本== | 高亮 | 这是 ==高亮内容== 。 |
| ^{^文本^} | 上标 | x^2 |
| _{~文本~} | 下标 | $a_{n~}$ |

格式标记内部会递归解析，因此不同结构可以嵌套：

包含 _强调内容_ 的粗体文字

分隔符匹配有意保持简单：每个结构在其结束分隔符第一次按字面出现的位置结束；分隔符不会进行平衡匹配，搜索时也不会跳过已转义字符。因此不支持相同分隔符的嵌套；复杂嵌套如果包含外层结束分隔符，也可能比预期更早结束。

任何已识别的开始分隔符都必须有对应的结束分隔符。未闭合的格式范围会报错，而不会被视为普通文本。

5.2 转义

反斜杠会使紧随其后的一个字符成为普通文本：

*字面星号
_字面下划线
\@字面 at 符号
\\字面反斜杠

任何字符都可以转义，并不仅限于标点。输入末尾单独出现的反斜杠会报错。

ProofText 转义只在行内解析时生效。在行内代码、行内公式、链接 URL、围栏代码块或展示公式中不会处理转义。转义也不会参与解析器最初对结束分隔符的搜索，因此已转义字符无法可靠地隐藏外层范围的结束分隔符。

使用普通 Lean 字符串时，不要忘记还要为 Lean 再转义一次反斜杠。Lean 原始字符串可以避免这一额外层次。

5.3 行内代码

一对反引号创建原样行内代码：

使用 ``provide h`` 闭合目标。

之后第一个反引号会结束代码范围。正文不会解析标记或转义。不支持使用多个反引号作为分隔符；缺少结束反引号会报错。

5.4 行内公式

一对美元符号创建行内 LaTeX：

等式 $x + 0 = x$ 成立。

之后第一个 $+$ 会结束该范围。正文原样保留，不会处理 ProofText 标记或转义。缺少结束 $+$ 会报错。

5.5 链接

链接使用以下形式：

[Proof-Script 仓库](https://github.com/example/Proof-Script)

标签会递归解析为行内 ProofText；URL 原样保留且不会进行有效性检查。解析器允许空标签和空 URL。

[之后第一个](用于分隔标签与 URL，随后第一个) 会结束链接。URL 中的括号不会进行平衡匹配，URL 内也不会处理 ProofText 转义。如果 [后没有](，它仍是普通文本；但已经识别出的链接开始标记如果缺少结束) ，则会报错。

5.6 标签

标题和引用使用以下形式的标签：

[A-Za-z][A-Za-z0-9_-]*

为保证与导出的 JSON Schema 兼容，应使用 ASCII 字母和数字。标签区分大小写。例如：

| 标签 | 是否合法 | 原因 |
|-------------|------|----------------|
| main-result | 是 | 以字母开头，允许连字符。 |
| result_2 | 是 | 首字母之后允许数字和下划线。 |
| 2-result | 否 | 以数字开头。 |
| main.result | 否 | 不允许点号。 |
| main result | 否 | 不允许空格。 |

5.7 引用

ProofText 有四种引用形式：

| 语法 | 目标种类 |
|--------------|-------------|
| @<label> | 标题 |
| @thm:label | 定理 |
| @fig:label | 图形或 TikZ 组件 |
| @table:label | 表格 |

例如：

参见 @<introduction> 和 @thm:main-result。
比较 @fig:overview-diagram 与 @table:measurements。

标题引用具有结束符 > 。带类型的引用形式会读取随后由字母、数字、连字符和下划线组成的最长序列。因此，@thm:result. 表示对 result 的引用，之后是普通标点。

一旦遇到已知引用标记，错误的引用语法就会报错。@eq:result 等未知形式仍作为普通文本。

解析引用时不会检查目标是否存在。引用可以指向后文，跨页面有效性则由之后的渲染器或使用方处理。

6. 识别规则与优先级

如果同一位置可能开始多种结构，ProofText 会使用固定的识别顺序。

在块起始行，实际顺序为：

1. 空行。
2. 以三个反引号开始的围栏代码块。
3. 行内容恰好为 $\$$ 时的展示公式。
4. 标题，同时拒绝在块位置以 = 开头的错误标题。
5. 以 - 开头的无序列表项。
6. 以 + 或十进制数字和 . 开头的有序列表项。
7. 以 > 开头的引用行。
8. 包含 :: 的定义项。
9. 段落。

在行内位置，顺序为：

1. 反斜杠转义。
2. 粗体强调 ***...***。
3. 删除线 ~~...~~。
4. 下划线 ...。
5. 高亮 ***...***。
6. 粗体 **...**。
7. 强调 ***...***。
8. 上标 ^{...}。
9. 下标 _{...}。
10. 行内代码 `...`。

- 11. 行内公式 `$...$` 。
- 12. 链接 `[...](...)` 。
- 13. 标题引用 `@<...>` 。
- 14. 定理、图形或表格引用。
- 15. 普通文本。

这是操作层面的优先级，而非平衡分隔符的优先级。解析器会先定位结束分隔符，之后才递归解析其中的正文。

7. 错误

以下错误形式会使 `#text` 命令在 `elaboration` 时失败：

| 情形 | 典型诊断信息 |
|--|---|
| 行内内容末尾仅有反斜杠 | dangling ProofText escape |
| 格式范围未闭合 | unclosed ProofText delimiter '...' |
| 行内代码未闭合 | unclosed inline code |
| 行内公式未闭合 | unclosed inline math |
| 已识别的链接缺少 <code>)</code> | unclosed ProofText link |
| 标题引用缺少 <code>></code> | unclosed heading reference |
| 标题引用标签错误 | invalid heading label '...' |
| <code>@thm:</code> 、 <code>@fig:</code> 或 <code>@table:</code> 后标签错误 | invalid reference after '...' |
| 围栏代码块未闭合 | unclosed code block |
| 展示公式未闭合 | unclosed display math |
| 块位置上的标题格式错误 | invalid heading or heading label; labels must match ... |
| 标题标签重复 | duplicate heading label '...' |

目前诊断信息会指向整个 Lean 字符串字面量，而不是 ProofText 内容中的精确偏移位置。

8. 当前限制

当前解析器尚未实现：

- 以 - 或 * 开头的无序列表；
- 多行列表项或嵌套列表项；
- 将连续定义项合并为一个语义定义列表；
- 平衡分隔符或相同分隔符的行内嵌套；
- 搜索结束分隔符时识别转义；
- 使用多个反引号的行内代码；
- URL 有效性检查或 URL 括号平衡；
- #text elaboration 阶段的引用目标检查；
- Markdown 硬换行语法或自动生成软换行、硬换行节点；
- 图片、HTML、水平分隔线、脚注或 ProofText 表格语法。

9. 速查表

块级结构

| 结构 | 语法 |
|-----------|------------------------|
| 段落 | 连续的普通非空行 |
| 标题 | = $\{1,6\}$ 标题 |
| 带标签标题 | = $\{1,6\}$ 标题 <label> |
| 无序列表项 | - 文本 |
| 自动编号有序列表项 | + 文本 |
| 强制序号有序列表项 | n. 文本 |
| 定义 | 术语 :: 定义 |
| 引用 | > 文本 |
| 展示公式 | 正文前后内容恰好为 \$\$ 的行 |
| 代码块 | 正文前后以 ``` 开头的行 |

行内结构

| 结构 | 语法 |
|------|---------------------------|
| 粗体 | *文本* |
| 强调 | <u>文本</u> |
| 粗体强调 | *_文本_* |
| 删除线 | 文本 |
| 下划线 | <u>文本</u> |
| 高亮 | ==文本== |
| 上标 | ^{文本} |
| 下标 | _{文本} |
| 代码 | <code>`代码`</code> |
| 公式 | $\$latex\$$ |
| 链接 | [标签](url) |
| 标题引用 | @<label> |
| 定理引用 | @thm:label |
| 图形引用 | @fig:label |
| 表格引用 | @table:label |
| 转义 | <code>\字符</code> |